

WORKSHEET 4: WORKFLOW MANAGEMENT WITH SNAKEMAKE

Kosmas Hench
2025-05-27

CONTENTS

1	The Problem	1
2	The Pieces	1
3	The Process	2
3.1	The Basics	2
3.2	First Steps	3
3.3	Extending a Workflow	5
3.4	Wrapping Up	8
3.5	Workflow Overview	9

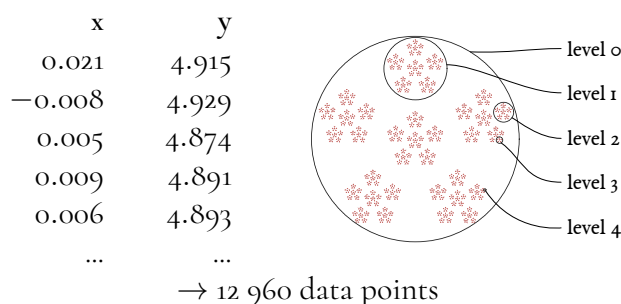
When encountering more complex analyses, one can often get lost in the sheer volume of the required code, even if it is well documented. And when trying to document your own code, it is very easy to gloss over some details or connections that seem irrelevant or obvious at the time the code is written (and the thinking behind its design is still fresh in the author's memory). Unfortunately, these small details later often turn out to be rather crucial knowledge, that is in fact far from obvious.

One way to ensure that all connections of an analysis are well documented and defined is to explain them to an agent that is incapable of taking anything for granted – to automatize the process in a way that it can be run a computer from start to end. We can do so by embedding our analysis in a workflow that manages the complete process from the raw data to the final figures and tables of an analysis.

1. THE PROBLEM

To illustrate the utility of a workflow manager, we are going to spin up another toy example: The basis of this exercise is going to be a simple data set, consisting of x and y values:

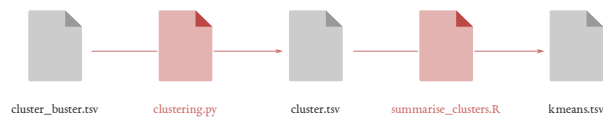
Overview of the `cluster_buster.tsv` data set



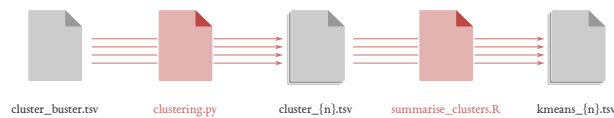
These data points are structured forming a **fractal** of nested clusters, that are each are arranged as sets of pentagons. Our task is going to be to descend into the depths of this fractal, and to try to assign the data points to separated clusters at different levels of this nested hierarchy. For this, we will revisit the kmeans-clustering approach from SESSION 1, and apply it repeatedly with varying parameter settings.

2. THE PIECES

The general idea for the analysis is not too complicated - in fact, we already did a lot of the legwork in the practical part of SESSION 1: again, we are going to use some `python` code to assign individual data points to separate clusters. Following this, we will use some `R` scripting to visualize the cluster assignment and to calculate some summary statistics per cluster:



Both provided scripts (`clustering.py` and `summarise_clusters.R`) build upon the code that we used for the analysis of the penguin data set earlier. Here, we add complexity by providing the option to run the cluster assignment and summary multiple times, each time assigning a different number of target clusters. The number of target clusters will depend on the level of the fractal that we are interested in: the clusters are grouped in sixes (five corners, plus central cluster), so on level one we will need to assign six clusters, on level two 36 (6×6), on level three 216 (6^3), and on level four 1296 clusters (6^4). Both, the python and R script, are thus written in a way that make them flexible and that we can specify the number of target clusters when running them. The overall process is always going to be the same, but we will need to execute it several times (once per number of target clusters).



We will wrap this task in a workflow, which will be managed by `snakemake`. This will allow us to automatize the execution of the repeated cluster assignment and to transparently document and communicate how often and with which settings each run was conducted.

3. THE PROCESS

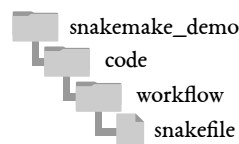
Today, we jumpstart the practical session by accessing the scaffold for our exercise from github:

```
git clone https://github.com/k-hench/snakemake_demo.git
```

Now, to get started with `snakemake` we will take it slowly. We will start with a very minimal *hello world* workflow and extend it bit by bit. This approach emulates the process you might encounter when designing an actual analysis in `snakemake`: there too, you will likely start simple, extending your analysis over time and slowly increase the complexity of your workflow.

3.1 The Basics

To get started, we need a `snakefile`, which should be located in the subdirectory `code/workflow` of your `project_folder`:



As `snakemake` is building upon `python`, this file is basically a special `python` script:

```

snakefile
1  """
2  snakemake -n
3  snakemake -c 1
4  """
5
6  # snakemake default target rule
7  rule all:
8      message:
9          '''
10         Hello. I am the default target of snakemake.
11         '''

```

The main building blocks of a **snakemake** workflow are so-called **rules**. These are individual tasks that need to be chained together to conduct the defined workflow. Typically, the logic for this chaining of rules is governed by the definition of **input** and **output** files that link individual **rules**.

If a workflow contains the **rule** called “all”, this will be the default target for **snakemake**. This means, that unless instructed otherwise, **snakemake** will try to complete all tasks necessary to successfully run the **rule** “all”. What exactly is required for this will depend on the files that are produced by “all”, and the dependencies for those.

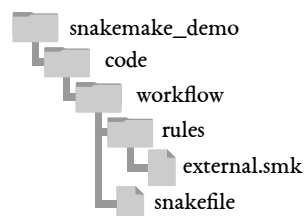
In the minimal example above, the **rule** does not specify **input** or **output** files. In this case, there is nothing to do (yet), and **snakemake** will simply prompt the specified message. This template also contains a comment in the header (lines 1–4), that provides the starting commands to run **snakemake** for convenience. The command **snakemake -n** will start a dry-run that will check what needs to be done without actually executing any code defined in the **rules**. This is useful to get a quick check of the coding syntax (detecting the most obvious bugs rather quickly). By specifying **-c 1** we run the workflow, reserving a single thread (1) of our computer for the execution.

```
cd snakemake_demo/code
snakemake -c 1
Job stats:
job      count  min threads  max threads
-----
all       1         1             1
total    1         1             1
Select jobs to execute ...
[<date> <time>]
Job 0:
  Hello. I am the default target of snakemake.
```

Cool, **snakemake** runs successfully, but as of now it does nothing...

3.2 First Steps

Now, let's fill the workflow with some life. For a workflow with a scope like in this example, we could keep it simple and pack all **rules** into the **snakefile** itself. But if you are working on more complex projects, you might want to bundle different steps of the analysis to not get lost. We can do this, by gathering the **rules** that belong to a specific branch of the analysis into external **.smk** files, which we place into the subfolder **workflow/rules**:



We bind these external **rules** into the workflow by importing the content of the **smk** files into the **snakefile**:

```
snakefile
1 """
2 snakemake -n
3 snakemake -c 1
4 """
```

```

5
6 # snakemake default target rule
7 rule all:
8     message:
9         '''
10         Hello. I am the default target of snakemake.
11         '''
12
13 # import rules managed in external scripts
14 # (for a clearer organization of the workflow)
15 include: "rules/clustering.smk"

```

Now, let's create `clustering.smk`, where we are going to set up the tasks for our cluster assignment:

```

rules/clustering.smk
1 """
2 snakemake -n step1
3 snakemake -c 1 step1
4 """
5 # --- defining "target rules" ---
6 rule step1:
7     input: "../data/cluster_buster.tsv"
8
9 # --- defining "worker rules" ---
10 rule create_data:
11     output:
12         tsv = "../data/cluster_buster.tsv",
13         pdf = "../results/img/cluster_generation.pdf"
14     shell:
15         '''
16         Rscript --vanilla R/cluster_buster.R
17         '''

```

Similarly to the `snakefile`, we start the `smk` files with a small header section (lines 1-4), providing the launch command for this part of the workflow.

Then we specify the “target rules”, containing only input files (lines 5-8). These are the desired products of this chunk of analysis. We can use the names of these rules to request `snakemake` to create the files specified for a particular target rule (s. commands in the header).

Finally, we provide the instructions to actually produce the desired files (lines 9-17). To begin, we need to run an R script that will create the `tsv` file `cluster_buster.tsv` in the `data` folder. It will also create a `pdf` file containing a plot of the data set.

Note that we refer to the R script as `R/cluster_buster.R`. So, make sure to copy the provided R script into that location:

```

mkdir R
cp ~/external_resources/scripts/R/cluster_buster.R R/

```

The path to the R script is specified in this form because the workflow is intended to be started from the code folder of the project. This will make `project_folder/code` the working directory for the `snakemake` session, and the path to the R script needs to be specified relative to this working directory.

For the same reason, the output files are prefixed with “`../`” (to get out of `project_folder/code` and into `data` and `results`).

→ At this point, we should be able to launch the workflow. Let's start by conducting a dry-run:

```

snakemake -n step1

```

Job stats:			
job	count	min threads	max threads
create_data	1	1	1
step1	1	1	1
total	2	1	1

According to the dry-run, we will need to complete two jobs to complete “step1”: one instance of the rule “create_data” and one instance of “step1”.

Looks good, so we can execute the workflow:

```
snakemake -c 1 step1
```

If all goes well, this should produce the desired results:

```
tree ../
```

```
├── code
│   ├── R
│   │   └── cluster_buster.R
│   └── workflow
│       ├── rules
│       │   └── clustering.smk
│       └── snakefile
├── data
│   └── cluster_buster.tsv
├── example.Rproj
├── readme.md
├── results
│   └── img
│       └── cluster_generation.pdf
```

Looks good, so let’s log the current progress to version control. However, we want to use `git` to track our *work*, not our *results*. So, add the `data` and the `results` folder to your `.gitignore` file first (also the hidden `.snakemake` folder, which contains the housekeeping for `snakemake`):

```
cd ../
echo -e "data/\nresults/" > .gitignore
echo "code/.snakemake/" >> .gitignore
git add .
git commit -m "starting snakemake workflow"
```

3.3 Extending a Workflow

With the data set in place, we can finally start implementing the actual analysis. As mentioned in the introduction, we will need a `python` script for the cluster assignment, and an `R` script for the cluster summary and plotting. So, let’s first import them from `deNBI`’s `external_resources` stash:

```
cd code
mkdir py
cp ~/external_resources/scripts/py/clustering.py py/
cp ~/external_resources/scripts/R/summarise_clusters.R R/
```

Next, we’ll tackle the fact that we want to run the analysis multiple times. We will refer to the individual iterations of the clustering analysis by the number of clusters that we assign the data points to (6×6 , 6^2 , 6^3 and 6^4). We prepare for this by creating the variable `n_clusters` that contains these values.

rules/clustering.smk

```
5 # --- setting up snakemake variables ---
6 # different expected number of clusters
7 # dependent on fractal-level
8 n_clusters = [ 6, 36, 216, 1296 ]
```

Note, this assignment of a variable is plain `python` coding inside a `snakemake` script – if you want to/ if necessary, you can get creative with your `python` skills more generally...

Then, we add two more target `rules` to run the workflow either to complete the cluster assignment, or to the completed cluster summary step.

In the `input` directives for these `rules`, we encounter ne new feature of `snakemake`:

Note, how the file names now contain a `{nclust}` tag. This is called a `wildcard` and represents a placeholder – in our case ro be replaced by the number of clusters to assign.

So, "`../results/summary/kmeans_{nclust}.tsv`" in fact stands representative for separate files (one per fractal level / number of clusters to assign). By wrapping this placeholder in the function `expand()`, we fill in the wildcard `{nclust}` with the values that we previously stored in the variable `n_clusters`.

This means that the rule “step3” requests four files: `kmeans_6.tsv`, `kmeans_36.tsv`, `kmeans_216.tsv` and `kmeans_1296.tsv`.

```
rules/clustering.smk
14 rule step2:
15     input: expand("../results/kmeans/cluster_{nclust}.tsv", nclust = n_clusters)
16
17 rule step3:
18     input: expand("../results/summary/kmeans_{nclust}.tsv", nclust = n_clusters)
```

We can (have to) use these in the output of the worker rules too – `snakemake` needs to be able to determine how the four files are to be created. Using this technique, we can specify that the rule “run_clustering” is flexible, and it can create *any* of the four `cluster_{nclust}.tsv` versions.

For this reason, we have created the `python` script so that it accepts the number of clusters to assign as a command line argument. Now we use this flexibility and pass the number of clusters from `snakemake` to the `python` script. To do so, we can access the value of `nclust` for each iteration of the rule and pass it to the code executed within the `shell` directive by referring to `{wildcards.nclust}`. Similarly, we are passing on the names of the necessary input and output files (referring to them as `{input.tsv}` and `{output.tsv}`, respectively).

```
rules/clustering.smk
30 rule run_clustering:
31     input:
32         tsv = "../data/cluster_buster.tsv"
33     output:
34         tsv = "../results/kmeans/cluster_{nclust}.tsv"
35     shell:
36         """
37         python py/clustering.py \
38             -n {wildcards.nclust} \
39             -i {input.tsv} -o {output.tsv}
40         """
```

It is possible to keep this ambiguity going for several steps, and to chain multiple rules that need to be run in succession for each iteration of this analysis part.

In our example, we first need to run the cluster assignment (`python` script), and then the cluster analysis (`R` script) for *each* number of clusters to assign (fractal level).

```
rules/clustering.smk
42 rule cluster_stats:
43     input:
44         tsv = "../results/kmeans/cluster_{nclust}.tsv"
45     output:
46         tsv = "../results/summary/kmeans_{nclust}.tsv",
47         pdf = "../results/img/assigned_{nclust}_clusters.pdf",
```

```

48     rds = "../results/img/R/assigned_{nclust}_clusters.Rds"
49 shell:
50     '''
51     Rscript --vanilla R/summarise_clusters.R {wildcards.nclust}
52     '''

```

At this point, `snakemake` should be ready to run the analysis. Let's check by conducting another dry-run, requesting the last target rule ("step3"):

```

snakemake -n step3
Job stats:
job              count
-----
cluster_stats    4
run_clustering   4
step3            1
total            9

```

Note that `snakemake` picks up where we left off: it realizes that we already have run a part of the workflow before, when we created the `cluster_buster.tsv` data set. Since we did not touch the source code for that part, `snakemake` does not plan to re-run anything that is already finished. This is great news, as we are avoiding unnecessary re-runs that way.

We can also see that `snakemake` now intends to run four instances of the "run_clustering" and "cluster_stats" rules. As the different iterations are independent from each other, we can speed up things by parallelizing the four iterations. We do so by allowing `snakemake` to use four threads when executing the latest additions (-c 4):

```
snakemake -c 4 step3
```

If all goes according to plan, we should now see the output as follows:

```

tree ../results/
../results/
├── img
│   ├── R
│   │   ├── assigned_1296_clusters.Rds
│   │   ├── assigned_216_clusters.Rds
│   │   ├── assigned_36_clusters.Rds
│   │   └── assigned_6_clusters.Rds
│   ├── assigned_1296_clusters.pdf
│   ├── assigned_216_clusters.pdf
│   ├── assigned_36_clusters.pdf
│   ├── assigned_6_clusters.pdf
│   └── cluster_generation.pdf
├── kmeans
│   ├── cluster_1296.tsv
│   ├── cluster_216.tsv
│   ├── cluster_36.tsv
│   └── cluster_6.tsv
└── summary
    ├── kmeans_1296.tsv
    ├── kmeans_216.tsv
    ├── kmeans_36.tsv
    └── kmeans_6.tsv

```

As always when we are happy with the progress of our project, it is a good idea to commit the current status to version control.

```
cd ..
git add .
git commit -m "added main cluster analysis"
```

3.4 Wrapping Up

To re-iterate the process, let's add one final step to the workflow. In this we will collect the output of the four iterations and bind them into a single summary report/ figure. Again, you can find the required R script for this in the `external_resources` folder:

```
cd code
cp ~/external_resources/scripts/R/meta_summary.R R/
```

Once more, we extend the `clustering.smk` file, providing the final target rule "completion":

```
rules/clustering.smk
20 rule completion:
21     input: "../results/img/cluster_summary.pdf"
```

Of course, we also need a worker rule that will execute the imported R script to produce the final figure. Here, we add the directive `params` to the rule. This can be used to pass any kind of parameters to the execution of a rule – in our case we forward the complete vector of possible cluster values from the `snakemake` variable `n_clusters` to the rule. Note that this rule is only executed once, and that all possible four values are passed one in one go (as a list containing four entries).

```
rules/clustering.smk
57 rule final_summary:
58     input:
59         tsvs = expand("../results/summary/kmeans_{nclust}.tsv", nclust = n_clusters)
60     output:
61         pdf = "../results/img/cluster_summary.pdf"
62     params:
63         nclusts = n_clusters
64     shell:
65         """
66         Rscript --vanilla R/meta_summary.R "{params.nclusts}"
67         """
```

Again, we conduct a try-run first to catch obvious bugs early in the process.

```
snakemake -n completion
Job stats:
job          count
-----
completion    1
final_summary  1
total         2
```

This task only requires a single execution of one job, so we can go back to `-c 1` to start `snakemake`. Fingers crossed, and hopefully we are done with designing our toy analysis.

```
snakemake -c 1 completion
tree ../results
../results/
├── img
│   ├── ...
│   └── cluster_summary.pdf
└── ...
```

```
cd ..
git add .
git commit -m "workflow completion"
```


3.5 Workflow Overview

Today’s example was still rather simple – real life cases can quickly become rather complex and hard to follow.

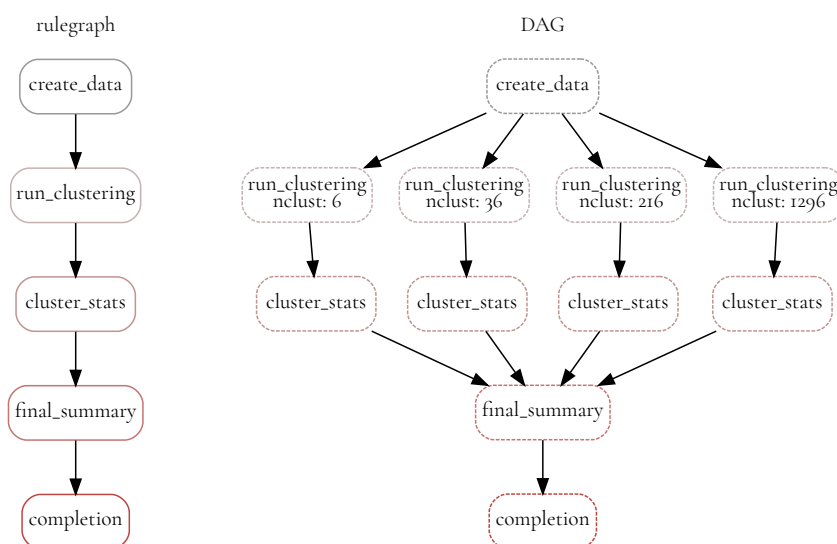
Particularly when adapting some else’s code (or coming back to you oooooold code) one can easily get lost while trying to trace bugs over multiple `smk` files. To ease this, and to provide some overview `snakemake` provides tools to visualize a workflow in different ways.

On a high level, one can create a “rulegraph” for a workflow, if a more explicit report is required, one can create a “DAG”. The difference is that the rulegraph will only display the general logic of the workflow, while the DAG traces all individual iterations of all required rules. In our example, this means that a rulegraph will only indicate that generally, one needs to assign clusters first (using the `python` script), to then summarize them (with the `R` script). In contrast, the DAG will explicitly list all four iterations of these two rules:

cd code

```
snakemake --rulegraph completion | dot -Tsvg > ../results/img/graph_completion.svg
```

```
snakemake --dag completion | dot -Tsvg > ../results/img/dag_completion.svg
```



Finally, we only scratched the surface of `snakemake` here. The goal was to provide a minimal but still non-trivial example to get everyone going. Most of the concepts that we have touched upon can be deepened by referring to the [snakemake manual](#)...